Инициализация, присваивание и объявление переменных

Оглавление:

1. Адреса и переменные
2. Фундаментальные типы данных в C++
3. Объявление переменных
4. Инициализация переменных
5. uniform-инициализация
6. Присваивание переменных
7. Объявление нескольких переменных
8. Где объявлять переменные?

Адреса и переменные

Как вы уже знаете, переменные — это имена кусочков памяти, которые могут хранить информацию. Помним, что компьютеры имеют оперативную память, которая доступна программам для использования. Когда мы определяем переменную, часть этой памяти отводится ей.

Наименьшая единица памяти — **бит** (англ. «**bit**» от «**b**inary dig**it**»), который может содержать либо значение `0`, либо значение `1`. Вы можете думать о бите, как о переключателе света — либо свет выключен (`0`), либо включен (`1`). Чего-то среднего между ними нет. Если просмотреть случайный кусочек памяти, то всё, что вы увидите, — будет `...01101010010101010...`, или что-то в этом роде. Память организована в последовательные части, каждая из которых имеет свой **адрес**. Подобно тому, как мы используем адреса в реальной жизни, чтобы найти определенный дом на улице, так и здесь: адреса позволяют найти и получить доступ к содержимому, которое находится в определенном месте памяти. Возможно, это удивит вас, но в современных компьютерах, у каждого бита по отдельности нет своего собственного адреса. Наименьшей единицей с адресом является **байт** (который состоит из 8 битов).

Поскольку все данные компьютера — это лишь последовательность битов, то мы используем **тип данных** (или просто «***тип***»), чтобы сообщить компилятору, как интерпретировать содержимое памяти. Вы уже видели пример типа данных: int (целочисленный тип данных). Когда мы объявляем целочисленную переменную, то мы сообщаем компилятору, что «кусочек памяти, который находится по *такому-то* адресу, следует интерпретировать как целое *число*».

Когда вы указываете тип данных для переменной, то компилятор и процессор заботятся о деталях преобразования вашего значения в соответствующую последовательность бит определенного типа данных. Когда вы просите ваше значение обратно, то оно «восстанавливается» из этой же последовательности бит.

Кроме int, есть много других типов данных в языке C++, большинство из которых мы детально рассмотрим на соответствующих уроках.

Фундаментальные типы данных в C++

В языке C++ есть встроенная поддержка определенных типов данных. Их называют **основными типами данных** (или ***«фундаментальные/базовые/встроенные типы данных»***).

Вот список основных типов данных в языке C++:

Категория	Тип	Значение	Пример
Логический тип данных	bool	true или false	true
Символьный тип данных	char, wchar_t, char16_t, char32_t	Один из ASCII-символов	'c'
Тип данных с плавающей запятой	float, double, long double	Десятичная дробь	3.14159
Целочисленный тип данных	short, int, long, long long	Целое число	64
Пустота	void	Пустота	

Объявление переменных

Вы уже знаете, как объявить целочисленную переменную:

```
int nVarName; // int - это тип, а nVarName - это имя переменной
```

Принцип объявления переменных других типов аналогичен:

```
type varName; // type - это тип (например, int), а varName - это имя переменной
```

Объявление пяти переменных разных типов:

```
bool bValue;
char cValue;
int nValue;
float fValue;
double dValue;
```

Обратите внимание, переменной типа void здесь нет (о типе void мы поговорим детально на следующем уроке).

```
void vValue; // не будет работать, так как void не может использоваться в качестве типа переменной
```

Инициализация переменных

При объявлении переменной мы можем присвоить ей значение в этот же момент. Это называется **инициализацией переменной**.

Язык C++ поддерживает 2 основных способа инициализации переменных.

Способ №1: **Копирующая инициализация** (или ***«инициализация копированием»***) с помощью знака равенства `=`:

```
int nValue = 5; // копирующая инициализация
```

Способ №2: **Прямая инициализация** с помощью круглых скобок `()`:

```
int nValue(5); // прямая инициализация
```

Прямая инициализация лучше работает с одними типами данных, копирующая инициализация — с другими.

uniform-инициализация

Прямая или копирующая инициализация работают не со всеми типами данных (например, вы не сможете использовать эти способы для инициализации списка значений).

В попытке обеспечить единый механизм инициализации, который будет работать со всеми типами данных, в C++11 добавили новый способ инициализации, который называется **uniform-инициализация**:

```
int value{5};
```

Инициализация переменной с пустыми фигурными скобками указывает на инициализацию по умолчанию (переменной присваивается `0`):

```
int value{}; // инициализация переменной по умолчанию значением 0 (ноль)
```

В uniform-инициализации есть еще одно дополнительное преимущество: вы не сможете присвоить переменной значение, которое не поддерживает её тип данных — компилятор выдаст предупреждение или сообщение об ошибке. Например:

```
int value{4.5}; // ошибка! целочисленная переменная не может хранить дробные значения
```

Правило: Используйте uniform-инициализацию.

Присваивание переменных

Когда переменной присваивается значение после её объявления (не в момент объявления), то это **копирующее присваивание** (или просто ***«присваивание»***):

```
int nValue;
nValue = 5; // копирующее присваивание
```

В языке C++ нет встроенной поддержки способов прямого/uniform-присваивания, есть только копирующее присваивание.

Объявление нескольких переменных

В одном **стейтменте** можно объявить сразу несколько переменных одного и того же типа данных, разделяя их имена запятыми. Например, следующие 2 фрагмента кода выполняют одно и то же:

```
int a, b;
```

И:

```
int a;
int b;
```

Кроме того, вы даже можете инициализировать несколько переменных в одной строке:

```
int a = 5, b = 6;
int c{7}, d{8};
int e{3}, f{10};
```

Есть 3 ошибки, которые совершают новички при объявлении нескольких переменных в одном стейтменте:

Ошибка №1: Указание каждой переменной одного и того же типа данных при инициализации нескольких переменных в одном стейтменте. Это не критичная ошибка, так как компилятор легко её обнаружит и сообщит вам об этом:

```
int a, int b; // неправильно (ошибка компиляции)

int a, b; // правильно
```

Ошибка №2: Использование разных типов данных в одном стейтменте. Переменные разных типов должны быть объявлены в разных стейтментах. Эту ошибку компилятор также легко обнаружит:

```
int a, double b; // неправильно (ошибка компиляции)

int a; double b; // правильно (но не рекомендуется)

// правильно и рекомендуется (+ читабельнее)
int a;
double b;
```

Ошибка №3: Инициализация двух переменных с помощью одной операции:

```
int a, b = 5; // неправильно (переменная b остается неинициализированной)

int a = 5, b = 5; // правильно
```

Хороший способ запомнить эту ошибку и не допускать в будущем — использовать прямую или uniform-инициализацию:

```
int a, b{5};
int c, d{5};
```

Этот вариант наглядно показывает, что значение `5` присваивается только переменным `b` и `d`.

Так как инициализация нескольких переменных в одной строке является отличным поводом для совершения ошибок, то я рекомендую определять несколько переменных в одной строке только в том случае, если вы будете каждую из них инициализировать.

Правило: Избегайте объявления нескольких переменных в одной строке, если не собираетесь инициализировать каждую из них.

Где объявлять переменные?

Более старые версии компиляторов языка Си вынуждали пользователей объявлять все переменные в верхней части функции:

```
#include <iostream>

int main()
{
    // Все переменные в самом верху функции
    int x;
    int y;

    // Спустя уже всех остальной код
    std::cout << "Enter a number: ";
    std::cin >> x;

    std::cout << "Enter another number: ";
    std::cin >> y;

    std::cout << "The sum is: " << x + y << std::endl;
    return 0;
}
```

Сейчас это уже неактуально. Современные компиляторы не требуют, чтобы все переменные обязательно были объявлены в самом верху функции. В языке C++ приоритетным является объявление (а также инициализация) переменных как можно ближе к их первому использованию:

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int x; // мы используем x в следующей строке, поэтому объявляем эту переменную здесь (как можно ближе к её первому использ
    std::cin >> x; // первое использование переменной x

    std::cout << "Enter another number: ";
    int y; // переменная y понадобилась нам только здесь, поэтому здесь её и объявляем
    std::cin >> y; // первое использование переменной y

    std::cout << "The sum is: " << x + y << std::endl;
    return 0;
}
```

Такой стиль написания кода имеет несколько преимуществ:

- Во-первых, возле переменных, которые объявлены как можно ближе к их первому использованию, находится другой код, который способствует лучшему представлению и пониманию происходящего. Если бы переменная `x` была объявлена в начале функции main(), то мы бы не имели ни малейшего представления, для чего она используется. Для понимания смысла переменной пришлось бы целиком просматривать всю функцию. Объявление переменной `x` возле операций ввода/вывода данных позволяет нам понять, что эта переменная используется для ввода/вывода данных.
- Во-вторых, объявление переменных только там, где они необходимы, сообщает нам о том, что эти переменные не влияют на код, расположенный выше, что делает программу проще для понимания.
- И, наконец, уменьшается вероятность случайного создания неинициализированных переменных.

В большинстве случаев, вы можете объявить переменную непосредственно в строке перед её первым использованием. Тем не менее, иногда, могут быть случаи, когда это будет не желательно (по соображениям производительности) или невозможно. Детально об этом мы поговорим на следующих уроках.

Правило: Объявляйте переменные как можно ближе к их первому использованию.